Introduction to Data Science

Assignment 5 100P

Submit by creating a pull request on GitLab.
Students of the Engineering Cohort can submit in Brightspace.

Please refer to and provide explanations, discussions and results in your report to present your solution to this assignment. Use as section headers the assignment part you provide the solution for and in them subsections or lists with a, b, etc. so we know which part you answer.

5.1 Reinforcement Learning (Example Exam Questions) (30P)

(a) (3/30P) **Supervised learning vs reinforcement learning**: **List three differences between supervised learning and reinforcement learning.**

(b) (3/30P) **Policies:** You programmed a robot to navigate in a maze only based on the information obtained by the front camera. It has the following policies: If a wall is directly in front turn 90 degree to the right. Otherwise move one step forward.

  (i) Is this a deterministic or a stochastic policy?

  (ii) Explain why you chose your answer.

(c) (1/30P) **An unsuccessful policy:** An engineer designed a reinforcement learning algorithm for a disaster robot. During the night, he wakes up because he suddenly realises that the policy, he implemented is going to be very unsuccessful in letting the robot move on the remaining of a broken building. However, the test department, which is located in another time zone already started the first test run. **Explain what will happen when the robot with the unsuccessful policy starts learning.**

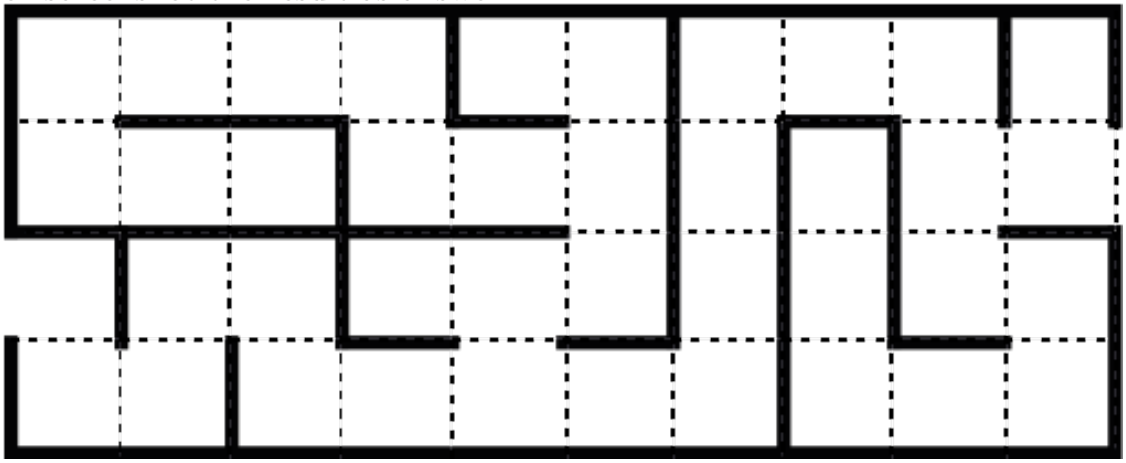
(d) (1/30P) **The Markov state:**
Explain which criteria need to be fulfilled so that a state is a Markov state.

(e) (4/30P) **Discount factor: Consider the extremes of discount factor values $\gamma = 0$ and $\gamma = 1$, what type behavior does an agent learn in each of these two cases?**

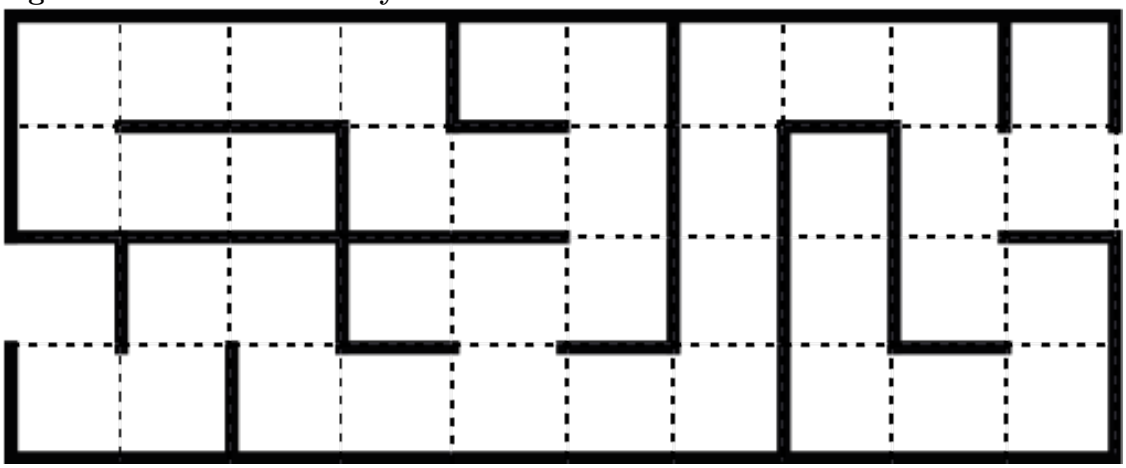
(f) (2/30P) **Like a mouse in a maze:** Consider training an agent to solve a maze. The environment punishes the agent with -1 for each step, and is in an end state when it finishes. Therefore the optimal reward is reached when the agent solves the maze in the fewest number of steps. Consider the agent is trained to decide to go left, right, up or

down based on states defined by the current position (x, y). **Argue whether it would be possible to train the agent to solve generic mazes, or only a particular one.**

(g) (8/30P) **Policy:** For the following maze the **entrance is right** and the **exit left**. Assume -1 reward per step. **Write in each cell the action of the most optimal policy, scan or screenshot the result as answer.**



(h) (8/30P) **Rewards:** Given this policy, also **write in each cell the cumulative reward. Again scan or screenshot your result to answer.**



5.2 A first reinforcement learning experiment (25P)

In this exercise we will use the package gym which provides environments for testing reinforcement learning. After installing the package use the supplementary material file `EnvTest.py` to test whether the package is working correctly. If everything went well you should see a little car that attempts to go uphill but does not manage to do so. If you see a black screen or nothing at all, you need to downgrade the version of the package so that it matches the python version that you are using.

In Matlab similar environments to the ones we are getting from gym are available in the reinforcement learning extension that is included in the license that is linked in Brightspace. These exercises have been designed in python and we recommend to use it to solve them.

(a) (2/25P) **Where is the policy?**
Is the policy that the car in the example `EnvTest.py` is currently follows stochastic or deterministic? Explain why?

(b) (4/25P) **Changing policies** Create two new versions of the example with the car. One of these should have a stochastic policy and one of them should follow a deterministic policy. Clearly indicate which example is the stochastic and which is the deterministic one.

(c) (3/25P) **From a continuous to a discrete problem** At the moment the example program does not learn anything. However, before you can start to implement your first Reinforcement Learning program you will need to discretize the state space as the problem at hand is continuous. What is the size of the discretized state space?

(d) (3/25P) **Initialize the Q-table:** After learning the Q-Table contains the expected future reward for an action at each state. Initialize the Q-table with the correct size. (Hint: if you were not able to solve the previous task use [19, 15] to initialize the states.)

(e) (3/25P) **Update the action based on the Q-table:** Use the Q-table you initialized to select the action based on the observed state. (Hint: states need to be discretized.)

(f) (5/25P) **Begin to learn:** Update the Q-table with a learning rate of 0.3 and a discount factor of 0.9. For this step you need to take the previous state action combination and the reward into account. (Hint: In the newer versions of gym the parameter done changes to true after a certain amount of attempts. If you would like your car to reach the top of the hill you need to add a termination condition yourself. The top of the hill is reached when the first parameter of the state exceeds 0.5. However, that might take a while. You can choose a smaller value if you are doing this exercise close to the hand in deadline.)

(g) (5/25P) **The epsilon greedy strategy** To get the car to explore solutions that it has not yet visited implement a greedy strategy. Use $\epsilon = 0.8$.

5.3 Learning takes time (45P)

(a) (4/45P) **Learning is repetition:** The car only gets a reward if the top is reached. To come to a good policy we need to visit this point multiple times. Implement a loop that allows you to learn multiple episodes (something between 100 and 1000 is reasonable). Make sure that you turn of the rendering. You can run it only for the last episodes. If you render all episodes your homework will keep your computer busy for quite a while and that is not necessary. Track the average reward per episode and plot it over the episodes.

(b) (10/45P) **The influence of the learning rate:**
Chose 5 different learning rates and explore how the behaviour of the algorithm changes depending on the learning rate.

(c) (6/45P) **The influence of epsilon:** Test 3 different values of epsilon and describe how that influence the result.

(d) (25/45P) **Don't drop in the water:** Implement your own reinforcement learning algorithm, that learns how to navigate the "FrozenLake-v1" environment of gym. You can use every type of algorithm that we had in the lecture. Do not use deep RL algorithms.